ANNALYSING THE IMPACT OF NEURAL NETWORK DEPTH ON LEARNING NON-LINEAR DECISION BOUNDARIES

STD_ID: 24118593

Github link: https://github.com/hibabtl/IMPACT-OF-NEURAL-NETWORK-DEPTH-ON-LEARNING-NON-LINEAR-DECISION-BOUNDARIES

1. INTRODUCTION:

Artificial Neural Networks (ANNs) are computational models inspired by the structure of human brain. They have become a cornerstone of modern machine learning due to their ability to model complex patterns in data. One of the most critical design choices in neural networks is their depth, which refers to the number of hidden layers in the model.

While deeper networks are often assumed to perform better, this is not always the case. Increasing depth can improve a model's ability to learn complex relationships, but it can also introduce challenges such as overfitting and increased computational cost.

This tutorial investigates how varying the depth of a neural network affects its performance. By comparing shallow, moderately deep and deep models, this study aims to demonstrate the trade-off between model complexity and generalisation.

2. METHODOLOGY:

2.1. Data Description:

The primary dataset used in this tutorial is the ***make_moons* dataset**, generated using Scikit-learn library. This dataset consists of two interleaving half-circle shapes representing two classes.

The dataset contains 1000 samples with two input features. A noise parameter of 0.2 is applied to simulate real-world imperfections. The dataset is suitable for this study because it is non-linearly separable, which means simpler linear models cannot effectively simplify it.

```
# Generate dataset
X, y = make_moons(n_samples=1000, noise=0.2, random_state=42)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

2.2. Model Design

Three neural network models are constructed using Tensorflow/Keras, each with a different depth:

- Model_1: 1 Hidden layer
- Model_3: 3 Hidden layers
- Model_5: 5 Hidden layers

All models used RELU activation function in hidden layers and sigmoid activation function in output layers for binary classification.

```python
def create_model(hidden_layers):
    model = Sequential()

    # Input layer
    model.add(Dense(hidden_layers[0], activation='relu', input_shape=(2,)))

    # Hidden layers
    for units in hidden_layers[1:]:
        model.add(Dense(units, activation='relu'))

    # Output layer
    model.add(Dense(1, activation='sigmoid'))

    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

    return model
```

```python
# Model A: Shallow
model_1 = create_model([8])

# Model B: Medium depth
model_3 = create_model([16, 16, 8])

# Model C: Deep
model_5 = create_model([32, 32, 16, 16, 8])
```

Model Architecture

2.3. Model Training:

Each model was compiled using the Adam optimizer and binary cross entropy loss function. Training was conducted for 50 epochs.

```python
history_1 = model_1.fit(X_train, y_train, epochs=50, validation_data=(X_test, y_test), verbose=0)
history_3 = model_3.fit(X_train, y_train, epochs=50, validation_data=(X_test, y_test), verbose=0)
history_5 = model_5.fit(X_train, y_train, epochs=50, validation_data=(X_test, y_test), verbose=0)
```

2.4. Evaluation Method:

Model performance was evaluated using validation accuracy. Graphs were plotted to compare accuracy of each model.

```
plt.plot(history_1.history['val_accuracy'], label='1 Layer')

plt.title("Validation Accuracy Comparison")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

3. RESULTS

3.1. MODEL_1 WITH ONE HIDDEN LAYER

The shallow model shows rapid initial learning but plateaued at approximately 0.85-0.87 validation accuracy. This indicates that the model lacks the capacity to fully capture the dataset complexity, resulting in underfitting as shown in the figure 3.1.1.

The decision boundary shown in figure 3.1.2 is relatively simple and almost linear, failing to follow the curved structure of the data.

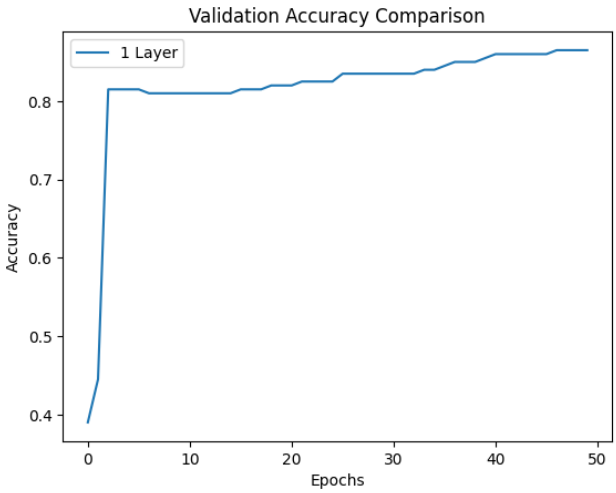

Figure. 3.1.1

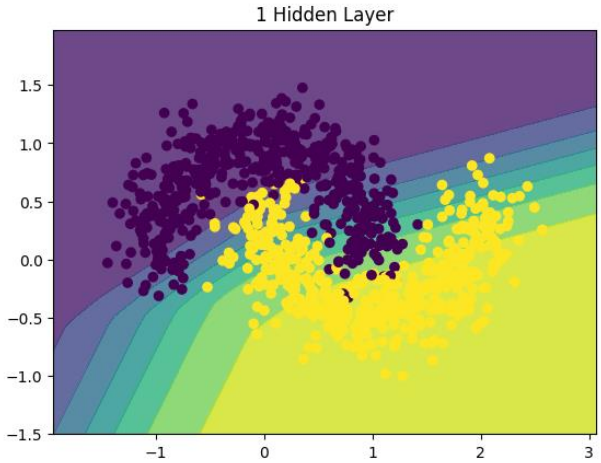

Figure. 3.1.2

3.2. MODEL_3 WITH THREE HIDDEN LAYERS

The three-layer model achieved the highest performance, reaching approximately 0.99 validation accuracy as shown in figure 3.2.1. The learning process was stable and consistent, indicating strong generalization.

The decision boundary closely follows the non-linear shape of the dataset, effectively separating the two classes.

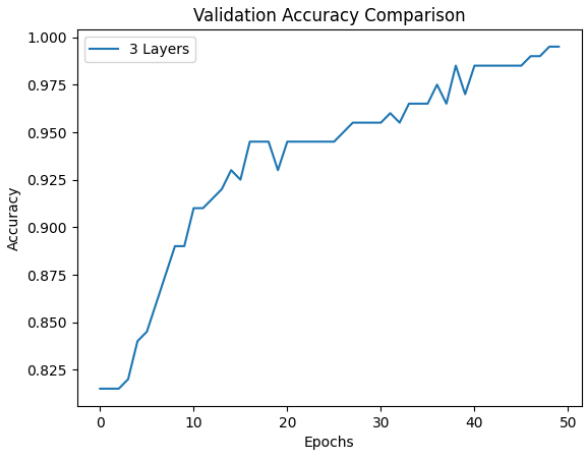

Figure.3.2.1

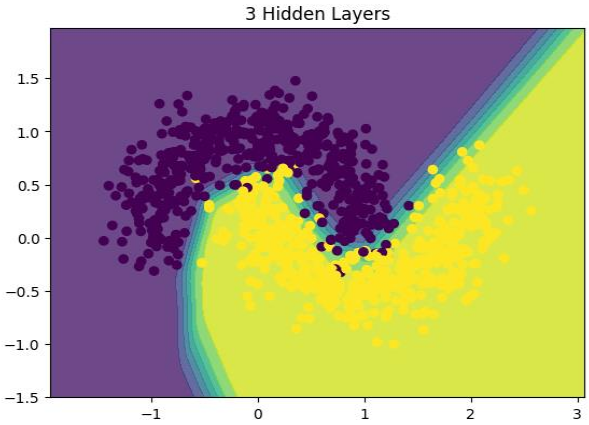

Figure.3.2.2

3.3. MODEL_5 WITH FIVE HIDDEN LAYERS

The five-layer model demonstrated higher complexity with accuracy approximately 0.98–0.99 but did not significantly outperform the three-layer model. Instead, it showed slight over fitting, where the model begins to capture noise as shown in figure 3.3.1.

The decision boundary is slightly complex than the three layer model but there is no significant difference between both as shown in figure 3.3.2.

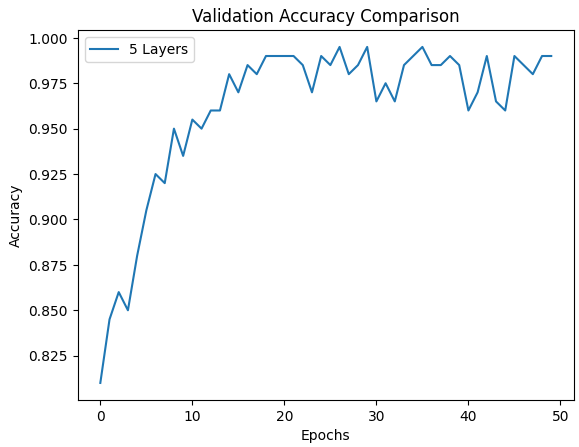

Figure 3.3.1

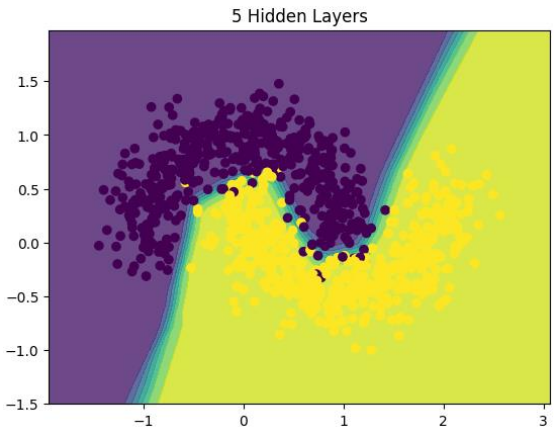

Figure 3.3.2

3.4. KEY INSIGHTS

The results demonstrate that increasing neural network depth improves performance to optimal point. Beyond this point, additional layers introduce unnecessary complexity and leading to overfitting.

4. DISCUSSION

The findings highlight the importance of selecting an appropriate network depth. While deeper models are more powerful, they are not always better. The three-layer architecture achieved the best balance between learning capacity and generalization.

A key practical question is how to determine the optimal number of hidden layers for the given tasks. In practice, there is no fixed rule; instead, it is determined experimentally.

One common approach is to start with a simple model and gradually increase the number of layers while monitoring the validation accuracy. The optimal depth is when the validation accuracy is maximised without significant fluctuations or decline.

In real world applications, selecting the right depth involves balancing performance, generalization and efficiency.

5. CONCLUSION

This tutorial explored the impact of neural network depth on model performance using the ***make_moons*** dataset. The results indicate that increasing depth enhances the model's ability to learn complex patterns, but only up to a certain point.

The three-layer model achieved optimal performance while shallow model underfits, and the deeper model showed slight signs of overfitting.

These findings emphasize the importance of carefully selecting model architecture based on the problem at hand. Depth should be treated as a tuneable parameter rather than a default choice.

REFERENCES

[1] Ian Goodfellow, Yoshua Bengio and Aaron Courville (2016) *Deep Learning*. MIT Press.

[2] Christopher M. Bishop (2006) *Pattern Recognition and Machine Learning*. Springer.

[3] Geoffrey Hinton (2006) "Reducing the dimensionality of data with neural networks." *Science*.

[4] Kaiming He et al. (2016) "Deep Residual Learning for Image Recognition." *CVPR*.

[5] TensorFlow Documentation. Available at: https://www.tensorflow.org

[6] Scikit-learn Documentation. Available at: https://scikit-learn.org

[7] Aurélien Géron (2019) *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*.

[8] Neural Networks and Deep Learning by Andrew Ng